Ephyra: A Property-Based Testing Framework for the LLM Era

ALPEREN KELES, Datadog, Inc., USA

Coding agents now write a growing share of software tests; however, property-based testing (PBT) libraries are designed with historical considerations. We present Ephyra, a PBT library in Rust built on three concepts: customizability, observability and generator synthesis. Ephyra lets users compose their own testing loops from generation, mutation and shrinking, and synthesize these components to satisfy functional predicates. Applying Ephyra to 14 open source systems, we have found 349 bugs and reported 300 of them, in database management systems (SQLite, Turso, PostgreSQL, DuckDB, DataFusion), the verified policy engine (Cedar), the SQL equivalence prover (QED), the URL parser used in Node.js (ada-url), reimplementations of existing open source systems in Rust (coreutils, brush, gitoxide), and compilers (binaryen, OCaml, and OxCaml).

1 Introduction

Large language models (LLMs) have significantly affected every phase of the software development life cycle (SDLC). Testing is no exception; the way software tests are read, written, and leveraged has been changing especially due to the growing popularity of spec-driven development [Amazon Web Services 2025] and the ability to run CEGIS-style [Solar-Lezama et al. 2006] program synthesis loops [Carlini 2026; Sumner 2026; Yang et al. 2026] relying on comprehensive test suites. This abstract addresses two questions—predominantly the latter question—in the midst of these changes:

(1) Should, and how can, we leverage LLMs in property-based testing workflows?
(2) How should the design of PBT libraries change with respect to the changing software engineering practices?

The first question is increasingly investigated in the literature and practice of PBT. The main interest is in using models to write property-based tests [Hegel developers 2026b; Maaz et al. 2025; Vikram et al. 2024] with further research on LLM-guided generation [Deng et al. 2023; Xia et al. 2024] and LLM-authored test-case generators [Chen et al. 2025; Vikram and Padhye 2026; Yang et al. 2024; Zhang et al. 2025b] and test case shrinkers [Zhang et al. 2024]. The second question, although naturally explored by using PBT libraries through agentic workflows, is currently under-researched. Similar questions have been posed in the context of dependency management [Ronacher 2025; Tomlinson 2026; Willison 2026] and data management systems domains [Liu et al. 2026]. In the context of PBT, the most significant effort to date is PBT skills [Hegel developers 2026a; Trail of Bits 2026] for LLMs. The core hypothesis behind the proposed library in §3 is two-fold. First, we posit that the future of property-based testing is hybrid. We hypothesize that instead of simply sampling random inputs from a constructed distribution, the future PBT workloads will interleave generation, enumeration and mutation depending on the domain and system under test. With this hypothesis, we develop a programmable PBT [Keles et al. 2026a] library that exposes primitives for constructing testing loops.

Second, program synthesis for verifiable domains has shown promising results with the growing capabilities of LLMs [Karpathy 2026; Novikov et al. 2025; Ouyang et al. 2025]. Existing work on generator derivation strategies includes type-based generators [Mista et al. 2018; Proptest contributors 2026], specification-based generators [Lampropoulos et al. 2018] derived from inductive specifications, and the randomness-parsing approach for deriving shrinkers and mutators from a given generator [Goldstein et al. 2023; MacIver 2026; MacIver and Donaldson 2020]. However, these approaches are incompatible with arbitrary functional predicates, and randomness-parsing derives specific shrinkers/mutators that are immutable. Our library includes a synthesis tool that

Author's Contact Information: Alperen Keles, Datadog, Inc., New York, USA, alp.keles@datadoghq.com.

can optimize generators for a given set of metrics while adhering to the validity predicates as well as correct-by-construction shrinker/mutator synthesis that is validated against the generator.

Under these hypotheses, we propose three design principles for our new library, Ephyra[1]:

(1) Verifiable components should be synthesized.
(2) Customizability should be a first-class concern.
(3) Observability should be a first-class primitive.

2 Background

This section presents existing literature and design space for the three principles outlined in §1.

2.1 Components in PBT

A *component* is a PBT abstraction, such as a generator, mutator, shrinker or printer, that defines an interface operating on the input type of the property. The classical property runner defined in QuickCheck [Claessen and Hughes 2000] can be decomposed into the following property runner diagram:

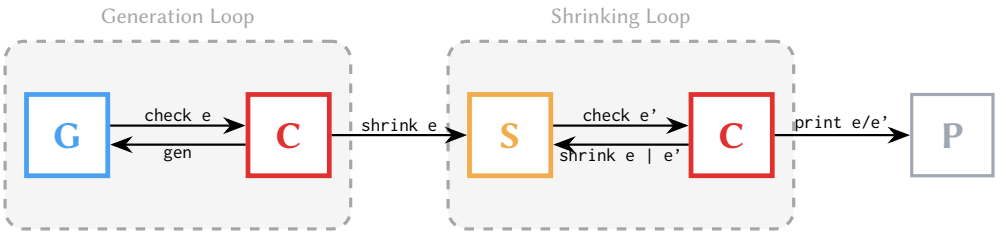


Fig. 1. Abstract representation of QuickCheck runner taken from Programmable PBT [Keles et al. 2026a].

The implementation of these components is a significant part of the PBT development; the quality of the test case generator has a significant effect on bug finding performance [Keles et al. 2026c], and further so, the design of the libraries themselves is affected by this effort trade-off. Many modern libraries use the "randomness-parsing" [MacIver et al. 2019] approach to test case generation, in which they generate random bytes that are then interpreted into structures, citing the benefit of this approach in automatically producing correct-by-construction shrinkers and mutators [MacIver and Donaldson 2020] without additional effort. Other libraries provide "derivations", compile-time code generation via macros to produce generators and shrinkers based on a type [Mista et al. 2018; Proptest contributors 2026], or in the case of QuickChick, inductive specifications [Paraskevopoulou et al. 2022]. Derivations are immensely useful in their convenience; derived generators can remove the hurdle of tediously writing and maintaining many small generators that could be trivial to write but time-consuming; derived shrinkers can be as good as the correct-by-construction shrinkers the randomness parsing libraries provide [Keles et al. 2026b]. Most of these derivations, however, are not long term testing strategies but starting points that lower the cost of property-based testing a new system. One notable exception is the derivation of generators that satisfy inductive specifications [Lampropoulos et al. 2018], giving rise to unlimited theoretical complexity, however limited in its applicability to testing programs due to the lack of such inductive specifications in practice. In Ephyra, we close this gap by "synthesizing" generators that satisfy functional specifications like the validity of a SQLite file (`PRAGMA integrity_check`) while optimizing the generator in a hill-climbing loop with respect to statistics we collect from the system under test.

[1] https://github.com/datadog-labs/ephyra

2.2 Customizability in PBT

A property-based testing framework typically provides a language for writing properties, a set of APIs for writing generators, automations for deriving generators and shrinkers from the input types, one or more testing loops with limited configurability for testing the properties by sampling the provided generators, and some observability hooks for measuring statistics computed from the tests. Programmable PBT [Keles et al. 2026a] is a new paradigm for PBT library design that gives the library users the primitives for constructing their own testing loops, a new axis in customization. The original work presents two design choices, (1) a property data structure in Rocq that carries a dependently-typed list of input values, and (2) a property data structure in Racket that carries a dynamically-typed context, both of which allow for inspecting the shape of the property. Leveraging the capability of inspecting the property, the authors then build four APIs—building blocks—for constructing existing property runners in the literature such as below:

```
Definition runLoop (fuel : nat) (cprop : Prop 0) :=
  let fix runLoop' (fuel : nat) (cprop : Prop 0)
                   (passed : nat) (discards : nat) : G Result :=
    match fuel with
    | O => ret (mkResult discards false passed [])
    | S fuel' =>
      input <- gen cprop (log2 (passed + discards));;
      res <- run cprop input ;;
      match res with
      | Normal seed false => (* Fails *)
        let shrunk := shrinker 10 cprop seed in
        let printed := print cprop 0 shrunk in
        ret (mkResult discards true (passed + 1) printed)
      | Normal _ true => (* Passes *)
        runLoop' fuel' cprop (passed + 1) discards
      | Discard _ _ => (* Discard *)
        runLoop' fuel' cprop passed (discards + 1)
      end
    end in
  runLoop' fuel cprop 0 0.
```

Fig. 2. QuickCheck-style property runner written in Rocq taken from Programmable PBT [Keles et al. 2026a].

The programmability of Ephyra is limited compared to the original design; we inherit the four building blocks `generate`, `mutate`, `shrink`, and `run` but keep the property representation opaque. This hybrid design allows for the users to write their own property runners, but not the building blocks themselves. For instance, the users can arbitrarily interleave generation and mutation, but never at the granularity of individual input variables, only for the whole set of input variables to the property. We present the capabilities of the library in §3.

2.3 Observability in PBT

The first question one must ask once they have written a property-based test is: *What does the test actually do?* The typical mechanisms PBT libraries provide for understanding the generators are (1) sampling a number of inputs from the generators, and (2) computing aggregate statistics via `collect` callbacks. Tyche [Goldstein et al. 2024] and the `OpenPBTStats` [Goldstein et al. 2024]

format have advanced the state of PBT observability by turning plain textual histograms computed by the libraries into dynamically rendered graphs while enriching the types of feedback libraries can emit during the testing runs. In Ephyra, observability, particularly through code coverage computed via LLVM coverage counters, is used extensively to guide the evolution of the generator. Generators are not treated as concrete programs that are written, but distributions to be shaped by the reviewers that rely on the observability to understand the underlying tests. We have built prototype interfaces for guiding the generators through the observations collected from the test runs, and we plan to keep developing them in the future.

3 Ephyra: A Programmable PBT Framework with Synthesizable Components

Ephyra, the ancient Greek city now known as Corinth, is the home of Sisyphus, a man forever cursed to carry a boulder up a hill, only to keep going in a metaphorical Möbius strip, never approaching the end. The name of the library is an homage to the most Sisyphean task in modern software development, testing; particularly random testing where we roll not one, but millions of rocks through a variety of software, and Ephyra is the home of all such efforts. The library follows the programmable PBT tradition of publishing existing property-runners; a simple generational runner, a fuzzing runner and a targeted runner are all within the current implementation. The core innovation, synthesized components, is managed via the accompanying command line interface, and constraints on the generators are denoted via Rust macros as shown in Fig. 3.

```
struct Canonical;
impl Gen for Canonical {
    type Value = String;
    fn generate(&self, g: &mut Source) -> String { ... }
    fn shrink(&self, s: &String, visit: ...) -> ControlFlow<()> { ... }
    fn mutate(&self, s: &String, g: &mut Source) -> String { ... }
}
// Validity predicate: ada parses the string to itself.
fn is_canonical(s: &String) -> bool {
    canonicalize(s).as_deref() == Some(s.as_str())
}
// Diversity metrics to optimize for the generator distributions
fn canonical_metrics(s: &String) -> Vec<(&'static str, f64)> {
    let b = |x: bool| f64::from(u8::from(x));   // true -> 1.0, false -> 0.0
    vec![("chars", s.chars().count() as f64), ("has query", b(s.contains('?'))), ...]
}

#[ephyra::generator(is_canonical, metrics = canonical_metrics)]
fn canonical_urls() -> impl Gen<Value = String> { Canonical }

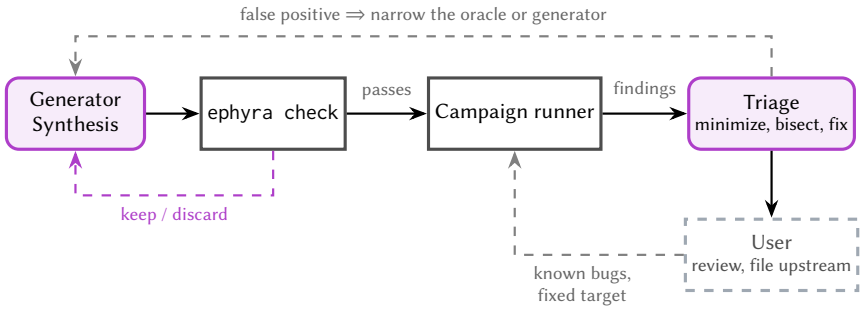
#[ephyra::test(gen = canonical_urls(), tests = 3000)]
fn canonical_url_is_reference_canonical(url: String) {
    let r = whatwg_url::run(&Case::parse(url.clone())); // reference implementation
    assert!(r.can_parse);
    assert_eq!(r.href(), Some(url.as_str()));
}
```

Fig. 3. Definition of a generator for the ada-url case study.

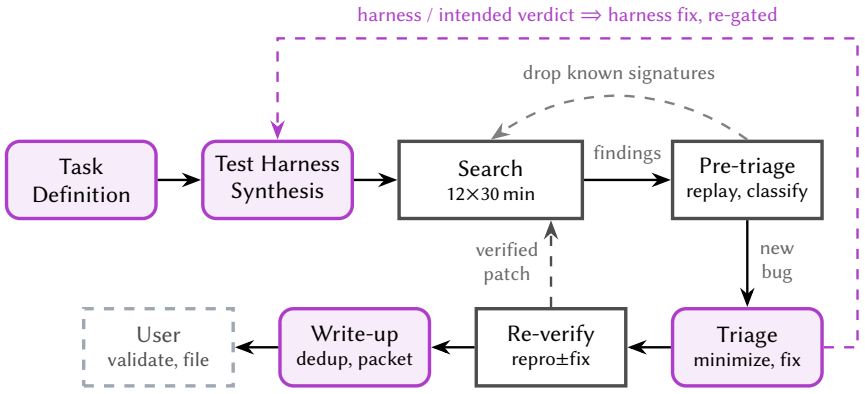
A validity predicate is an arbitrary Rust function of type `fn(&T) -> bool`, generators mark both their validity predicates as well as the set of metrics to be computed over the distribution of the inputs. For each annotated generator, `ephyra check` performs a full Ephyra run over 10,000 inputs by default and verifies, per input, that (1) the generated value satisfies the predicate; (2) the first 64 shrink candidates differ from the value and satisfy the predicate; (3) repeatedly taking the first shrink candidate stays valid, never revisits a value, and settles within 1000 steps; (4) four mutants stay valid; and (5) every metric is finite. We use the result of these checks to drive the component synthesis for repairing broken generators, mutators and shrinkers as well as improving accepted ones against the set of computed metrics, complemented by code coverage.

4 Preliminary Experimental Results

We applied Ephyra to fourteen real systems in two modes that differ in the steering of the generator synthesis. Appendix A lists every bug with its upstream report. In the *open-loop* style, we have run interactive coding-agent sessions that write Ephyra generators, predicates and properties for one target, tune the generators against a metric, run campaigns, and triage what they find. In the *closed-loop* style, the steering judgement is left up to `ephyra-agent` with automated patches within the loop to strengthen the ability to find new bugs.



(a) Open loop: interactive session



(b) Closed loop: the ephyra-agent controller

Fig. 4. Two experiment modes: rounded, tinted boxes are LLM agents; square boxes are deterministic tools; the dashed grey box is a user.

4.1 Open-Loop Case Studies

We ran open-loop studies on six open-source systems: SQLite (63), Turso (72), PostgreSQL (16), ada-url (12), Cedar (2), and QED (2); 167 confirmed bugs in total. We have used classic properties from the literature such as the metamorphic database testing oracles [Rigger and Su 2020a,b], simpler model implementations for model-based testing and differential testing against reference implementations. The ability to generate against arbitrary predicates has given us the opportunity to experiment with new styles of generators, such as directly generating SQLite files from scratch as opposed to the classic approach of running multi-round stateful tests against the database.

4.2 Closed-Loop Campaign

The closed-loop experiment ran ephyra-agent (Figure 4b) on five Rust and C++ systems in parallel, for about eight hours, one 16-CPU workspace and a $100 budget each. Table 1 lists, per target, the number of verified bugs, the region coverage of the target's source code, and the agent spend. After the initial campaign, we kept running it informally, finding 72 more bugs; a newer trial on compilers has led to 30 bugs and 11 issues in binaryen, OCaml, and OxCaml.

Table 1. Closed-loop campaign. Setup (intake and harness), triage and write-up are agent costs in $.

Target	Property	Bugs	Cov.	Setup ($)	Triage ($)	Write-up ($)	Total ($)
uutils coreutils	differential vs. GNU coreutils	30	33.4%	11	50	25	87
brush	differential vs. bash	17	35.7%	13	42	12	67
gitoxide	differential vs. git	14	27.3%	11	24	11	45
DuckDB	TLP, NoREC, optimizer toggles	11	22.4%	20	41	12	74
DataFusion	TLP, NoREC, rule toggles	8	34.9%	12	37	9	57
Total		80		67	195	68	330

5 Discussion and Future Work

The main contribution we presented in the experimental results is not unique to Ephyra; it is possible to set up any PBT library with a similar optimization loop, and intuitively expect similar results. We see our experiments as a starting point for agent-authored optimized generators, and Ephyra's design choices as a good set of primitives for such explorations. Evaluations are immensely more complex to design when LLMs are part of the evaluated system; the classical design of ETNA [Keles et al. 2026c] was not sufficient for our measurements, so we plan to look into existing coding agent evaluation frameworks such as Harbor [Harbor Framework Team 2026] for more controlled experimental settings that only modify the generator, keeping the rest of the system constant to understand the effects in isolation. One of the crucial problems we encountered was reward hacking. Given a metric, an agent can optimize solely the metric rather than the testing power it stands for. Designing metrics that resist gaming, and detecting the reward hacking remains an open problem for agent-driven generator synthesis. The future work we imagine is multifaceted: we would like to work on turning Tyche [Goldstein et al. 2024] into an interactive tool for shaping generators instead of just understanding them; instead of simply measuring the generators, we would like to prove properties about their soundness and completeness with respect to the desired distribution; we would like to explore the effects of different property-runners in more detail and understand how we can utilize source-level structured mutations more effectively than we do today. Finally, the optimization architecture we built is a very simple hill-climbing loop. Researchers have shown a variety of different strategies for autonomous optimization loops [Lange et al. 2025; Novikov et al. 2025; Romera-Paredes et al. 2023; Zhang et al. 2025a] that we plan to evaluate on generator optimization and adopt when we observe their benefits.

References

Amazon Web Services. 2025. Kiro. https://kiro.dev/. Accessed 2026-10-20.

Nicholas Carlini. 2026. Building a C compiler with a team of parallel Claudes. https://www.anthropic.com/engineering/building-c-compiler. Anthropic engineering blog, February 5, 2026. Accessed 2026-10-20.

Chuyang Chen, Brendan Dolan-Gavitt, and Zhiqiang Lin. 2025. ELFuzz: Efficient Input Generation via LLM-driven Synthesis Over Fuzzer Space. In *34th USENIX Security Symposium (USENIX Security 25)*. USENIX Association, Seattle, WA, 6279–6298. https://www.usenix.org/conference/usenixsecurity25/presentation/chen-chuyang

Koen Claessen and John Hughes. 2000. QuickCheck: a lightweight tool for random testing of Haskell programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP)*. ACM, New York, NY, USA, 268–279. doi:10.1145/351240.351266

Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. 2023. Large Language Models Are Zero-Shot Fuzzers: Fuzzing Deep-Learning Libraries via Large Language Models. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 423–435. doi:10.1145/3597926.3598067

Harrison Goldstein, Samantha Frohlich, Meng Wang, and Benjamin C. Pierce. 2023. Reflecting on Random Generation. *Proceedings of the ACM on Programming Languages* 7, ICFP (2023), 322–355. doi:10.1145/3607842

Harrison Goldstein, Jeffrey Tao, Zac Hatfield-Dodds, Benjamin C. Pierce, and Andrew Head. 2024. Tyche: Making Sense of PBT Effectiveness. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology (UIST)*. ACM, 1–16. doi:10.1145/3654777.3676407

Harbor Framework Team. 2026. Harbor: A Framework for Evaluating and Optimizing Agents and Models in Container Environments. https://github.com/harbor-framework/harbor. doi:10.5281/zenodo.20953922 Accessed 2026-10-20.

Hegel developers. 2026a. hegel-skill: An Agent Skill for Writing Hegel Tests. https://github.com/hegeldev/hegel-skill. Accessed 2026-10-20.

Hegel developers. 2026b. hegel-zoo: Property-Based Tests for Open-Source Projects, in Every Language Hegel Supports. https://github.com/hegeldev/hegel-zoo. Accessed 2026-10-20.

Andrej Karpathy. 2026. autoresearch: AI Agents Running Research on Single-GPU nanochat Training Automatically. https://github.com/karpathy/autoresearch. Accessed 2026-10-20.

Alperen Keles, Justine Frank, Ceren Mert, Harrison Goldstein, and Leonidas Lampropoulos. 2026a. Programmable Property-Based Testing. arXiv:2602.18545 [cs.SE] https://arxiv.org/abs/2602.18545

Alperen Keles, George Miao, and Leonidas Lampropoulos. 2026b. Evaluating Shrinking (Experience Report). arXiv:2608.09935 https://arxiv.org/abs/2608.09935 To appear in Haskell '26: Proceedings of the 19th ACM SIGPLAN International Haskell Symposium.

Alperen Keles, Jessica Shi, Nikhil Kamath, Tin Nam Liu, Ceren Mert, Harrison Goldstein, Benjamin C. Pierce, and Leonidas Lampropoulos. 2026c. Etna: An Evaluation Platform for Property-Based Testing. *Journal of Functional Programming* 36 (2026). doi:10.46298/jfp.17867

Leonidas Lampropoulos, Zoe Paraskevopoulou, and Benjamin C. Pierce. 2018. Generating Good Generators for Inductive Relations. *Proceedings of the ACM on Programming Languages* 2, POPL (Jan. 2018), 30 pages. doi:10.1145/3158133

Robert Tjarko Lange, Yuki Imajuku, and Edoardo Cetin. 2025. ShinkaEvolve: Towards Open-Ended And Sample-Efficient Program Evolution. arXiv:2509.19349 https://arxiv.org/abs/2509.19349

Shu Liu, Soujanya Ponnapalli, Shreya Shankar, Sepanta Zeighami, Alan Zhu, Shubham Agarwal, Ruiqi Chen, Samion Suwito, Shuo Yuan, Ion Stoica, Matei Zaharia, Alvin Cheung, Natacha Crooks, Joseph E. Gonzalez, and Aditya G. Parameswaran. 2026. Supporting Our AI Overlords: Redesigning Data Systems to be Agent-First. In *Conference on Innovative Data Systems Research (CIDR)*. https://www.vldb.org/cidrdb/papers/2026/p32-liu.pdf arXiv:2509.00997.

Muhammad Maaz, Liam DeVoe, Zac Hatfield-Dodds, and Nicholas Carlini. 2025. Agentic Property-Based Testing: Finding Bugs Across the Python Ecosystem. arXiv:2510.09907 https://arxiv.org/abs/2510.09907 NeurIPS 2025 Workshop on Deep Learning for Code.

David R. MacIver. 2026. Hypothesis, Antithesis, synthesis. https://antithesis.com/blog/2026/hegel/. Antithesis blog, March 24, 2026. Introduces Hegel (https://hegel.dev/). Accessed 2026-10-20.

David R. MacIver and Alastair F. Donaldson. 2020. Test-Case Reduction via Test-Case Generation: Insights from the Hypothesis Reducer (Tool Insights Paper). In *34th European Conference on Object-Oriented Programming (ECOOP 2020) (LIPIcs, Vol. 166)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 13:1–13:27. doi:10.4230/LIPIcs.ECOOP.2020.13

David R. MacIver, Zac Hatfield-Dodds, and Many Contributors. 2019. Hypothesis: A New Approach to Property-Based Testing. *Journal of Open Source Software* 4, 43 (2019), 1891. doi:10.21105/joss.01891

Agustín Mista, Alejandro Russo, and John Hughes. 2018. Branching Processes for QuickCheck Generators. In *Proceedings of the 11th ACM SIGPLAN International Symposium on Haskell*. ACM, 1–13. doi:10.1145/3242744.3242747

Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. 2025. AlphaEvolve: A Coding Agent for

Scientific and Algorithmic Discovery. arXiv:2506.13131 https://arxiv.org/abs/2506.13131

Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. 2025. KernelBench: Can LLMs Write Efficient GPU Kernels? arXiv:2502.10517 https://arxiv.org/abs/2502.10517

Zoe Paraskevopoulou, Aaron Eline, and Leonidas Lampropoulos. 2022. Computing Correctly with Inductive Relations. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*. ACM, 966–980. doi:10.1145/3519939.3523707

Proptest contributors. 2026. proptest-derive: Custom-Derive for the Arbitrary Trait of proptest. https://docs.rs/proptest-derive/latest/proptest_derive/. Version 0.9.0. Accessed 2026-10-20.

Manuel Rigger and Zhendong Su. 2020a. Detecting Optimization Bugs in Database Engines via Non-Optimizing Reference Engine Construction. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 1140–1152. doi:10.1145/3368089.3409710

Manuel Rigger and Zhendong Su. 2020b. Finding Bugs in Database Systems via Query Partitioning. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020), 1–30. doi:10.1145/3428279

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. 2023. Mathematical discoveries from program search with large language models. *Nature* 625, 7995 (2023), 468–475. doi:10.1038/s41586-023-06924-6

Armin Ronacher. 2025. Build It Yourself. https://lucumr.pocoo.org/2025/1/24/build-it-yourself/. Blog post.

Armando Solar-Lezama, Liviu Tancau, Rastislav Bodík, Sanjit Seshia, and Vijay Saraswat. 2006. Combinatorial Sketching for Finite Programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 404–415. doi:10.1145/1168857.1168907

Jarred Sumner. 2026. Rewriting Bun in Rust. https://bun.com/blog/bun-in-rust. Bun blog, July 8, 2026. Accessed 2026-10-20.

Jacob Tomlinson. 2026. Open Source Libraries Are Evolving into Reference Implementations. https://jacobtomlinson.dev/posts/2026/open-source-libraries-are-evolving-into-reference-implementations/. Blog post, August 11, 2026. Accessed 2026-10-20.

Trail of Bits. 2026. Property-Based Testing Skill, Trail of Bits Skills. https://github.com/trailofbits/skills/tree/main/plugins/property-based-testing. Accessed 2026-10-20.

Vasudev Vikram, Caroline Lemieux, Joshua Sunshine, and Rohan Padhye. 2024. Can Large Language Models Write Good Property-Based Tests? arXiv:2307.04346 [cs.SE] https://arxiv.org/abs/2307.04346

Vasudev Vikram and Rohan Padhye. 2026. Fuzzing with Agents? Generators Are All You Need. arXiv:2604.01442 https://arxiv.org/abs/2604.01442

Simon Willison. 2026. My answers to the questions I posed about porting open source code with LLMs. https://simonwillison.net/2026/Jan/11/answers/. Blog post.

Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2024. Fuzz4All: Universal Fuzzing with Large Language Models. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. ACM, 1–13. doi:10.1145/3597503.3639121

Chenyuan Yang, Yinlin Deng, Runyu Lu, Jiayi Yao, Jiawei Liu, Reyhaneh Jabbarvand, and Lingming Zhang. 2024. WhiteFox: White-Box Compiler Fuzzing Empowered by Large Language Models. *Proceedings of the ACM on Programming Languages* 8, OOPSLA2 (2024), 709–735. doi:10.1145/3689736

John Yang, Kilian Lieret, Jeffrey Ma, Parth Thakkar, Dmitrii Pedchenko, Sten Sootla, Emily McMilin, Pengcheng Yin, Rui Hou, Gabriel Synnaeve, Diyi Yang, and Ofir Press. 2026. ProgramBench: Can Language Models Rebuild Programs From Scratch? arXiv:2605.03546 https://arxiv.org/abs/2605.03546

Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. 2025a. Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents. arXiv:2505.22954 https://arxiv.org/abs/2505.22954

Kunpeng Zhang, Zongjie Li, Daoyuan Wu, Shuai Wang, and Xin Xia. 2025b. Low-Cost and Comprehensive Non-textual Input Fuzzing with LLM-Synthesized Input Generators. arXiv:2501.19282 https://arxiv.org/abs/2501.19282 34th USENIX Security Symposium.

Mengxiao Zhang, Yongqiang Tian, Zhenyang Xu, Yiwen Dong, Shin Hwei Tan, and Chengnian Sun. 2024. LPR: Large Language Models-Aided Program Reduction. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 261–273. doi:10.1145/3650212.3652126

A Bug Reports

Table 2 lists the 349 bugs found in the case studies of §4: 167 in the open-loop studies, 80 in the closed-loop campaign, 72 in a later closed-loop run, and 30 in a later trial on compilers. *Duplicate* means an existing upstream issue or pull request already covers the bug; *won't report* means we

decided not to report the bug; *not yet reported* means its write-up is ready but not filed; *declined* means upstream closed our report as not planned.

Table 2. Bugs found by the case studies, with their upstream reports.

Bug	Report	Status
SQLite (open loop, 63)		
ROLLBACK TO after DROP TABLE/INDEX commits a stale root page (incremental vacuum)	forum 3bc3c8adfa	fixed
DELETE leaves an unaccounted byte on an interior page (WITHOUT ROWID, 3-byte key)	–	fixed
x IN (SELECT ...) in HAVING or the result converts the GROUP BY term x	–	fixed
Blob over 4000 bytes read from overflow pages is converted as UTF-8 in a UTF-16 database	forum 48c64d8bab	fixed
LEFT JOIN with b = NULL AND c IN (...) on index (b, c) also returns a matched row	forum 78bc86a164	fixed
SELECT * over a USING join left of a RIGHT/FULL JOIN fails with ambiguous column name	forum 8be1215e2c	reported
Flattened UNION ALL subquery with a RIGHT/FULL JOIN after its first SELECT loses WHERE terms	forum d098617140	fixed
Window query drops the left row of a correlated LEFT JOIN whose ON has a bare outer column	forum 7f4ae01b58	fixed
UNION ALL subquery/CTE with inner join left of a RIGHT/FULL JOIN returns only its last arm	forum 50dd78a44b	fixed
GROUP BY/ORDER BY over a subquery ordered the other direction skips the outer sort	forum 5cd4b3b27a	fixed
json_valid() returns 0 for canonical JSON made from JSON5 input (stale cached parse)	forum 1339f93b28	fixed
Join drops rows with an expression index over a column pinned by col = constant	forum 529732a774	fixed
MULTI-INDEX OR over a partial index WHERE a=const fails with internal query planner error	forum 3ede79ce42	fixed
Partial index WHERE col = constant makes col read back as the constant with col's affinity	forum 35992eec46	partly fixed
Bloom filter before an INTEGER PRIMARY KEY lookup drops the REAL join key -2^63	forum 403ca95409	fixed
Constant propagation leaves REAL -2^63 a REAL and misses an INTEGER -2^63 row	forum 9bbaa34eea	partly fixed
sum(b ORDER BY a) aggregates the ORDER BY term instead of the argument	forum b53a65d1fc	fixed
TEXT over 4000 bytes read from overflow pages loses a leading BOM in a UTF-16 database	forum 36a73e8c8a	fixed
JoinMiss optimization reuses a pruned automatic index across correlated subquery runs	forum 8e5c07be86	fixed
LIKE/GLOB optimization's two-range index scan is treated as ordered under IN/skip-scan	forum 8cbaf327c7	fixed
Planner ignores NOT INDEXED on WITHOUT ROWID tables (plan only)	forum cd41fceaa0	fixed
x IN (SELECT y COLLATE NOCASE ...) seeks a BINARY index and misses rows	forum f505c400cf	fixed
ADD COLUMN blob DEFAULT is read as UTF-8 when cast to text in a UTF-16 database	forum a9c654a346	fixed
Index on an untyped VIRTUAL generated column over REAL returns integral REALs as INTEGER	forum 6973ad169e	fixed
WITHOUT ROWID ORDER BY added column NULLS LAST with PK IN: malformed error or wrong rows	forum eee345f931	fixed

Bug	Report	Status
group_concat() window drops a separator after an inverse step leaves the text empty	forum b02f0a01e0	fixed
WITHOUT ROWID row-value range into an added column seeks the PK: malformed or missing rows	forum eee345f931 (reply)	reported
concat()/concat_ws() take a BLOB's size before UTF-8 conversion in a UTF-16 database	forum c621e963a9	duplicate
Row-value IN list driving a join reuses the full IN's ephemeral table, repeating rows	forum 764df92da2	fixed
Row-value IN (SELECT a, b) over WITHOUT ROWID with added column b probes the PK: malformed	forum eee345f931 (reply)	reported
Compound with ORDER BY COLLATE applies the last SELECT's GROUP BY/HAVING/DISTINCT to all	forum 3859ed06b6	fixed
OR in a LEFT JOIN's ON with the no-op join table omitted: wrong rows or SIGSEGV	forum 16c7c8892b	fixed
Btree delete keeps the deleted cell's cached CellInfo; a later write reports malformed	forum 3e17701ba5	fixed
Compound merge removes duplicates under one operand's collation, not the compound's	forum a3d1c48f31	fixed
RENAME COLUMN of a TEMP table rewrites temp triggers and views twice, breaking them	forum c75ba2bcd0	fixed
FTS5 ft = 'q': constant propagation breaks auxiliary functions (no such cursor)	forum a27cfbee6c	reported
FTS5 MATCH with rowid = non-integer returns every matching row	forum 422031d9f4	fixed
Virtual table created in a savepoint skips xRollbackTo; FTS transaction then fails	forum 9454b36e0a	fixed
FTS5 detail=none/column tokenizes UNINDEXED columns; snippet() reports malformed	forum a3eee51a91	fixed
Geopoly rowid = non-integer value finds the truncated rowid; UPDATE/DELETE hit the wrong row	forum 6d40716f60	fixed
FTS5 locale=1, contentless_unindexed=1 with an UNINDEXED column: every INSERT is malformed	forum c7ab2c9250	reported
FTS5 tokendata=1 query for a term missing from a segment reads another page: corruption error	forum 8a9e1dca7e	fixed
FTS5 contentless_unindexed table: DROP leaves %_content behind and RENAME skips it	forum 87d4bdefc0	fixed
Geopoly UPDATE of the rowid without _shape fails with an invalid polygon error	forum 1b9d323951	fixed
R-Tree UPDATE SET rowid / INSERT with rowid ignore the new rowid	forum 26da7cd9e2	fixed
FTS5 secure-delete: stale doclist-index jump returns rows of the next term†	–	fixed
FTS5 detail=none: secure-delete of a rowid replaced with secure-delete off corrupts the index	forum e380b6876c	reported
fts5vocab term range constraints and ORDER BY term use UTF-8 order in a UTF-16 database	forum b0c03988e8	reported
Compound rows change with ORDER BY when a later member sets the collation (since 3.53)	forum 2fb4734a27	reported
FTS5 secure-delete: a merge drops a delete marker, so a later secure delete revives old entry†	forum 737cb495ea	reported
DROP CONSTRAINT of a foreign key with SET DEFAULT/NOT DEFERRABLE/MATCH fails or leaves part†	forum 5e80e94512	fixed
DROP NOT NULL removes NOT DEFERRABLE instead, or drops a DEFERRABLE clause after NOT NULL†	forum 28af38d7c3	fixed
FTS5: phrases of a NEAR group that does not match a row report stale instances for it	forum 9fa08e2dbd	reported
FTS5: wrong instances below a non-matching AND, and right of a NOT whose left side is empty†	forum 5cffabd8e9	reported

Bug	Report	Status
fts5vocab rowids restart in every scan, so MULTI-INDEX OR and RIGHT JOIN return wrong rows	forum d29f636f37	reported
RENAME COLUMN fails on an INTEGER PRIMARY KEY column named in a UNIQUE table constraint	forum b088eb5b02	reported
MULTI-INDEX OR on a WITHOUT ROWID table misses rows when an OR term uses OP_SeekScan	forum f554fe3656	fixed
FTS5 trigram: LIKE and GLOB on an UNINDEXED column return no rows	forum be50ab5692	reported
WHERE OR on a table left of a RIGHT JOIN drives a MULTI-INDEX OR and is never tested	–	won't report
FTS5 tokendata=1: prefix query then term query in an open transaction reads freed memory	forum 5c53f75fbb	fixed
FTS5 trigram LIKE pushdown reads one byte past the pattern (sqlite3Fts5ExprPattern)†	forum 402c7177cb	fixed
In a UTF-16 database a zeroblob() used as text is converted from UTF-8, so CAST(zeroblob(4) AS TEXT) has 8 bytes and depends on the plan	forum 420911fdac	fixed
UPDATE re-applies affinity to unassigned columns: REAL -2^63 becomes INTEGER in the table but not in untouched indexes (trunk regression)	–	won't report
Turso (open loop, 72)		
X -> X / X -» X with identical operands panics (index out of bounds)	#9836	reported
Date/time modifiers accept trailing text	#9837	fixed
json_each/json_tree with a NULL root path errors instead of returning no rows	#9838	reported
json_each/json_tree error on an integer beyond i64 instead of returning a real	#9839	fixed
Negative-zero REAL becomes -0.0 in JSON; SQLite writes 0.0	#9840	reported
json_error_position at end of input after whitespace: internal error or off by one	#9841	fixed
Unused CTE is resolved and its errors reported when the statement has any subquery	#9842	fixed
json_patch with a repeated key (object then null) gives malformed JSONB or wrong text	#9843	reported
json_set/jsonb_set/json_remove mishandle NULL results, inputs and paths	#9844	reported
->/-» abbreviated, 32-bit integer and BLOB paths return NULL	#9845	reported
RIGHT/FULL JOIN with a WHERE subquery over a table-valued function panics	#9847	fixed
MATERIALIZED CTE with a self-referencing subquery in a nested query: internal error	#9848	fixed
Text-to-real parsing for REAL affinity and sum/avg differs from CAST beyond exponent ~99	#9849	reported
RIGHT JOIN then LEFT JOIN whose ON subquery reads the left table: panic or wrong rows	#9850	reported
Subqueries and constants hoisted before the loop raise errors SQLite never hits	#9851	reported
FULL/RIGHT JOIN onto a WITHOUT ROWID table with a secondary index panics	#9852	fixed
HAVING terms on GROUP BY columns are not applied before aggregating	#9880	fixed
Hash join returns a whole-number REAL stored as an integer as an INTEGER	#9881	fixed
MATERIALIZED CTE first filled inside a nested subquery's loop panics	#9882	fixed
min/max window with a frame and outer ORDER BY rowid fails with table not found	#9883	fixed
strftime() ignores the julian-day range check	#9884	fixed
json_pretty of an empty object/array adds a newline	#9885	reported
timediff skips day>28 normalization	#9886	fixed

Bug	Report	Status
HAVING predicate that is a GROUP BY term reads the column after the scan	#9887	fixed
strftime %J / REAL formatting prints 17 digits	#9888	reported
-0.0 in a REAL column keeps its sign	#9889	fixed
+HH:MM modifier with hour 24 adds a day	#9921	fixed
json_insert at the root path replaces the document	#9924	fixed
-9223372036854775808 and -9.223372036854776e18 treated as the same expression	#9925	fixed
BETWEEN on a partial index's column panics (covering index misjudged)	#9926	fixed
JSON infinity rendering by global string replace corrupts JSON strings	#9927	fixed
UPDATE through a covering index with a rowid-reading partial index: panic or duplicate rowid	#9932	fixed
UPDATE with a BEFORE UPDATE trigger builds an index key without REAL affinity: corrupt index	#9933	fixed
Partial index + join panics: column not stored in the index	#9934	fixed
Date/time functions don't stop at an embedded NUL	#9935	fixed
JSON subtype lost when a result column contains a GROUP BY expression	#9966	fixed
JSON cache treats a REAL 1.0 as the INTEGER 1 seen just before	#9967	fixed
Date/time functions reject a trailing vertical tab after the time value	#9968	fixed
strftime() prints an overflowing day as is instead of normalizing it	#9969	fixed
CREATE INDEX rejects double-quoted string literals SQLite accepts	#10337	reported
likelihood() rejects a float literal with an exponent (625e-4) or in parentheses	#10338	reported
pragma_quick_check / pragma_integrity_check name their column message	#10339	reported
ANALYZE cannot reach the temp schema or TEMP tables and indexes	#10340	fixed
VACUUM INTO ':memory:' writes a file named :memory:; VACUUM INTO '' rejected	#10341	fixed
json_each/json_tree declare column types, so comparisons apply the wrong affinity	#10342	fixed
JSON5 // comment ends only at LF, not at CR, U+2028 or U+2029	#10343	fixed
group_concat over a sliding window frame panics or drops an empty string	#10352	reported
Column qualified with sqlite_master not found when FROM names sqlite_schema	#10353	reported
PRAGMA index_xinfo reports the rowid entry's name as '' instead of NULL	#10354	reported
ANALYZE on a database without tables does not create sqlite_stat1	#10355	reported
soundex() returns '?000' for input with a non-letter	#10356	reported
json_patch/jsonb_patch decode escapes in added keys; jsonb_patch writes a malformed key	#10370	reported
json_error_position one character off for a bad escape or a number without digits	#10378	reported
RENAME COLUMN of a WITHOUT ROWID primary-key column moves it to the end of the key	#10387	reported
Correlated subquery reads NULL for a hash-build table when the probe side is a LEFT JOIN	#10388	reported
subquery not found for a WHERE subquery reading a table outside a materialized hash build	#10389	reported
RIGHT JOIN unmatched-rows pass panics (cursor is None) with a nested FROM subquery	#10390	reported
ADD COLUMN ... DEFAULT -9223372036854775808 reads as REAL for existing rows	#10391	reported
STRICT type check runs before NOT NULL, so INSERT OR IGNORE fails on a row SQLite skips	#10392	reported
UPSERT DO UPDATE is compiled when the INSERT cannot conflict; its errors fail the INSERT	#10393	reported

Bug	Report	Status
concat_ws() drops BLOB arguments and returns NULL when the separator is a BLOB	#10394	reported
Compound ORDER BY x ISNULL does not match a result column written x IS NULL	#10395	reported
schema.alias.column naming an outer query's table is not found in a subquery	#10396	fixed
PRAGMA table_info/table_list keep a view's old columns after its table changes	#10397	reported
sqlite_schema.sql holds a re-serialized statement instead of the original text	#10398	reported
PRAGMA index_list/index_info/index_xinfo omit a WITHOUT ROWID table's primary-key index	#10399	fixed
unhex(X, Y) returns NULL when Y is an INTEGER, REAL or BLOB	#10400	fixed
UPDATE of a child table fails with foreign key mismatch when no FK column is assigned	#10401	fixed
replace(X, '', NULL) returns NULL instead of X	#10426	fixed
OR over an index and the rowid misses rows: a seek leaves an earlier DeferredSeek pending	–	not yet reported
JSON functions take a non-JSONB BLOB for JSONB and skip leading whitespace bytes	–	not yet reported
UPDATE on a parent table fails with foreign key mismatch while foreign keys are off	–	not yet reported
PostgreSQL (open loop, 16)		
Nested BETWEEN grows exponentially in parse analysis and ignores statement_timeout	BUG #19758	reported
Hashed set operations and recursive UNION pre-size hash tables without the hash_mem cap	BUG #19759	reported
CLUSTER/REPACK drops a live row updated in a rolled-back subtransaction	BUG #19760	reported
btree_gist date index: inserting infinity fails with cannot subtract infinite dates	BUG #19761	reported
EXPLAIN ANALYZE of a Parallel Append sharing a SubPlan fails an assert in shm_toc_insert	BUG #19766	reported
ORDER BY/GROUP BY of a negated constant deparses as a column position; dumps do not restore	BUG #19767	reported
GROUP BY with ROLLUP and ndistinct statistics fails with corrupt MVNDistinct entry	BUG #19772	reported
Cross-partition UPDATE of a self-referencing table with ON UPDATE CASCADE fails with 23503	BUG #19781	reported
SET CONSTRAINTS in a released savepoint survives ROLLBACK TO an outer savepoint	BUG #19782	reported
Sorted GROUPING SETS leave a grouping column un-nulled (PG 19 regression)	BUG #19783	reported
min/max optimization costing = ANY over max(col) fails with no relation entry for relid 1	BUG #19784	reported
JSON_TABLE: subquery in PASSING fails with XX000; PASSING evaluated once per column	BUG #19786	reported
could not find pathkey item to sort: Gather Merge over merge join below reordered LEFT JOINs	BUG #19787	reported
MCV statistics with a domain-over-bool column: expression not found in statistics object	BUG #19789	reported
XX000 cannot start a subtransaction with unprocessed inval messages after ALTER DOMAIN	BUG #19790	reported
ALTER COLUMN TYPE of an INCLUDE column of an index backing a foreign key fails (2BP01)	BUG #19791	reported

Bug	Report	Status
ada-url (open loop, 12)		
Drive-letter reference against a file: base marks the URL as having an opaque path	ada#1278	reported
url_search_params keeps ill-formed UTF-8 instead of decoding it to U+FFFD	ada#1279	declined
URLPattern test/exec/match resolve a relative string input against an empty base URL	ada#1280	fixed
URLPattern special-scheme pathname does not canonicalize a backslash to /	ada#1281	fixed
set_host(host:port) over the max input length applies the host, drops the port, returns true	ada#1282	reported
canonicalize_protocol indexes an empty string_view for an escaped-colon protocol pattern	ada#1283	reported
URLPattern opaque pathname: a leading // is parsed as an authority and dropped	ada#1284	reported
URLPattern search canonicalization uses the non-special query encode set	ada#1285	reported
url_aggregator host setters exceed the max input length with IDNA-expanding hosts	ada#1326	reported
generate_pattern_string drops braces around a part whose prefix starts with / but is longer	ada#1327	fixed
IDNA combining-mark, bidi and normalization tables predate the Unicode 17 mapping table	idna#97	fixed
x86-64 builds run SSSE3 instructions without a runtime CPU check	–	fixed
Cedar (open loop, 2)		
upsert_entities keeps a stale ancestor when the batch names the same entity twice	cedar#2606	fixed
add_entities compares parents with the stored closure: drops parents, refuses identicals	cedar#2629	reported
QED (open loop, 2)		
Real constants parsed as f32, so distinct literals are proved equal (wide ones panic)	qed#13	reported
Panic on INTEGER vs DECIMAL operands in IN (subquery) and UNION	qed#14	reported
uutils coreutils (closed loop, 45)		
comm compares lines including their newline, rejecting sorted input	#15439	fixed
comm --check-order --total still prints the total after an order failure	#15254	duplicate
cksum with a legacy algorithm omits the file name for an explicit -	#15254	duplicate
csplit --suppress-matched rejects a line number one past the end of input	#15584	fixed
csplit /regex/ after a line-number pattern never tests the line the split stopped at	#15440	reported
expand rejects --tabs=2,/2 and --tabs=4,+2 as not ascending	#15579	fixed
dd status=none still warns that it cannot skip on a short stdin	PR #13115	duplicate
fmt puts one space instead of two after a sentence-ending first word at line end	#15585	reported
expand misplaces tab stops after a file without a trailing newline and past 8 KiB	#15656	reported
fold counts control characters such as ESC and DEL as zero columns	#15571	reported
join orders a missing join field before an empty one	#15254	won't report
join treats an empty or blank-only line as one empty field	#15254	won't report
nl pads unnumbered lines with width+1 spaces, ignoring the -s length	#15591	reported
od -S with -Ax prints 7-digit hex offsets instead of 6	#15592	fixed
sort -V ignores -i, -d and -f	#15587	reported
seq -w pads to the width of the increment, which is never printed	#15572	fixed
sort ignores a key's own r when the key is the whole line (-k1r)	#15570	fixed

Bug	Report	Status
sort treats form feed and carriage return as blanks	#15588	fixed
numfmt --invalid: one invalid field leaves the whole line unconverted	#15575	reported
split --verbose prints nothing with -n/--number	#15583	reported
split accepts a suffix start too large for the suffix length on empty input	#15569	reported
join: trailing blanks start an extra field; -i prints the first line's join field	#15576	fixed
split -a 0 with -n N uses a 1-character suffix instead of the default	#15581	fixed
tr validates string2 constructs against the wrong mode	#15589	reported
numfmt --to=si/iec ignores --round for values that get no suffix	#15574	reported
sort -h ignores -f, so lowercase suffixes are not folded	#15577	reported
expand counts a NUL byte as one column, so the next tab stop comes early	#15580	reported
nl resets the header/footer section to body at each input file	#15586	fixed
numfmt --round changes the last digit of a number that needs no rounding	–	won't report
sort: global -i is not overridden by -d	#15578	reported
basenc --base64url -d rejects input whose last group has no = padding*	#15664	reported
cat -s drops a single blank line after a line continued from an earlier file or read*	#15655	reported
chmod rejects modes such as -=r, -+x and -,u+x as unknown options*	#15665	reported
chmod --reference processes the first FILE operand last*	#15666	reported
csplit hangs on /REGEX/-N after a line-number pattern that stopped at a regex line*	#15653	reported
cut --output-delimiter prints the last field twice when the line has no terminator*	#15654	reported
dd conv=swab pairs bytes within each input block, so output depends on bs= and reads*	#15646	reported
numfmt prints -0 for a negative number that rounds to zero*	#15661	reported
printf %e/%g round an exact halfway value up instead of to even*	#15662	reported
printf %.*s accepts a * precision above 2147483647 instead of failing*	#15663	reported
split -l parses its count like a byte SIZE (-l 010 means 8 lines, -l 1k accepted)*	#15660	reported
expand/unexpand take a bare - operand as an obsolete -N tab option, skipping stdin*	#15658	reported
unexpand handles a line continuing from one FILE into the next as two lines*	#15657	reported
uniq -f treats carriage return and form feed as field separators*	#15659	reported
unexpand counts a NUL byte as one column, so blanks after it hit the wrong tab stops*	PR #15627	duplicate
brush (closed loop, 27)		
+= on a declare -i variable panics on i64 overflow (overflow-checked builds)	#1577	reported
${s/pat/rep}: $1, $0, ${name} in the replacement are read as regex group references	–	fixed
Negative subscript on a sparse array counts from the element count, not the highest index	#1570	fixed
printf invalid-number messages ignore 2> redirection, and the exit status is 0	#1589	reported
break/continue do not track how many loops enclose them	#1595	reported
Unexpanded ~01 loses its leading zero and prints as ~1	#1592	reported
Error inside a compound command with 2> ignores that redirect	#1596	reported
Command substitution hangs when a builtin writes over 64 KiB after a nested $( )	#1576	reported
Glob ""* (empty quotes before the leading dot) does not match dot files	#1584	reported
An error on one line of a multi-line $( ) does not end the command substitution	#1593	reported
${v:=word} expands to word, not v's new value, under declare -u/-l/-c/-i	#1585	reported
Brace expansion is not applied to assignment arguments of declare/local/export	#1569	reported
A shell error in one pipeline stage aborts the parent shell	PR #1538	duplicate

Bug	Report	Status
${a[@]+x} with an empty array gives one empty word instead of none	#1568	fixed
~- expands to an empty string when OLDPWD is declared but unset	#1591	reported
An assignment error inside a function does not abandon the rest of the calling line	#1594	reported
nullglob removes a non-pattern word like [a/b] (bracket expression containing /)	#1582	reported
echo -n -n hi fails with a usage error, and -e wins over a later -E*	#1588	reported
[[-z == x]] is accepted as a string comparison; bash rejects it*	#1587	reported
shopt -s globstar is accepted, but ** does not match recursively*	#1581	reported
printf with a large * width aborts and drops the rest of the output*	#1590	reported
read drops control characters from a pipe or file and treats ^C/^D as keystrokes*	#1579	reported
declare -f indents lines inside multi-line strings and here-documents*	#1580	reported
Glob path component without metacharacters keeps backslashes from an expansion*	#1583	reported
${v/pat/rep}: unquoted & in the replacement is not replaced by the match*	#1586	reported
A syntax error later in a script or -c string stops every earlier command from running*	#1578	reported
(case a in a) echo y;; esac) is a syntax error: esac) parsed as another case item*	PR #1529	duplicate
gitoxide (closed loop, 31)		
@{push} fails when the branch has no configured remote; git falls back to origin	#3096	reported
@{u}@{1} and @{push}@{...} are rejected with unconsumed input	#3098	reported
rev-parse of a short name fails when an earlier DWIM candidate is a dangling symref	#3090	reported
<name>@{n} fails when the branch name looks like a short hash	#3099	reported
@{-N} peels an annotated tag that git returns unpeeled	#3095	reported
<ref>@{n} on an unborn HEAD or dangling symref returns a reflog entry; git fails	#3089	reported
@{-1} resolves to a deleted branch whose name matches a dangling symref	#3090	reported
SHA-256 repositories: loose objects are never found by an abbreviated id	#3101	reported
<rev>^{/regex} on an unpeelable ref (unborn HEAD) searches all refs instead of failing	#3100	reported
<ref>@{n} resolves to the null object id when the entry records a ref deletion	#3089	reported
A short object prefix fails when a dangling symref has the same name	#3090	reported
<ref>@{0} returns the newest reflog entry instead of the ref's current value	#3089	reported
<ref>@{N} fails when the reflog has exactly N entries (oldest has a previous value)	#3089	reported
@{push} with push.default=upstream and a separate push remote maps through the wrong remote	#3096	reported
Tree merge debug-asserts: one side renames and edits a file, the other makes the same edit*	#3091	reported
Commit::describe() ignores annotated tags that point to another tag*	#3103	reported
topo traversal returns a tip that an end hides (Parents::First, or tip also an end)*	#3105	reported
rev_walk().with_hidden() returns ancestors of hidden commits under skewed commit times*	#3104	reported
<rev>:file/ resolves to the blob; git rejects a trailing slash on a non-directory*	#3102	reported
Tree merge keeps the ancestor when one side edits a file, the other makes it a symlink*	#3107	reported
Tree merge: same content on both sides plus one-sided chmod +x keeps the ancestor*	#3097	reported

Bug	Report	Status
status() fails when HEAD's branch is a symbolic ref to an unborn branch*	#3112	reported
status() misses a staged deletion when the index entry is intent-to-add*	#3115	reported
status() lists a file that replaced a submodule as untracked as well as a type change*	#3113	reported
Tree merge reports a false dir/file conflict when the other side deletes part of the dir*	#3106	reported
status ignores a gitlink whose path is not listed in .gitmodules*	#3114	reported
status() reports an executable replaced by a symlink as Modified, not TypeChange*	#3111	reported
Tree merge loses a file renamed onto a directory that the other side modified*	#3109	reported
Tree merge treats d as renamed to y after a move of d/e to y/z*	#3108	reported
Text merge reports no conflict when both sides append different lines at the end*	#3110	reported
Tree merge drops a file added on both sides when only one side makes it executable*	#3097	reported
DuckDB (closed loop, 29)		
Outer join with ON-clause subqueries under an inequality join: DataChunk::Slice assertion	#27717	reported
SEMI join ON true replaced by a cross product leaves a stale projection map (bind error)	#27452	reported
Nested loop join ignores the projection map after an outer join with an ON subquery	#27717	reported
Piecewise merge join ignores the projection map after an outer join with an ON subquery	#27717	reported
dense_rank() over an empty local sort collection segfaults	#27448	fixed
Hash join with a zero-column probe side fails: tuple width should not be 0	#27450	fixed
Join order optimizer drops a relation-free filter (WHERE NULL returns a row)	#27451	fixed
Statistics propagation table filter loses a column reference (assertion)	#27447	reported
Join elimination asserts on a conditionless join from a correlated aggregate subquery	#27445	reported
Join elimination leaves a stale parent projection map (missing child column)	#27449	reported
Statistics propagation folds an aggregate with no expressions (assertion exprs.size() > 0)	#27446	fixed
SEMI JOIN on a right-only condition returns no rows with filter_pushdown disabled*	#27633	reported
Caching-operator flush continues into a finished operator: join_hashtable assertion*	#27722	reported
LEFT JOIN ON false AND x IN (SELECT ...) fails: Failed to bind column reference*	#27719	reported
Join with an ungrouped aggregate over an empty table: LIMIT 1 returns 3 rows*	#27714	reported
Table filter found to be true-or-NULL keeps rows where the predicate is NULL*	#27715	reported
Lateral reference in a FULL JOIN ON subquery: INTERNAL !ContainsDependentJoin*	#27716	reported
CROSS JOIN + LEFT JOIN ON (SELECT true) + WHERE: assertion "normalized" in join order*	#27723	reported
x IN (<correlated scalar subquery>, x) treated as NULL-propagating: drops a row*	#27712	reported
FULL JOIN with an ON subquery over a CROSS JOIN: Vector::Reference type mismatch*	#27726	reported
abs() statistics on FLOAT/DOUBLE are backwards when the range ends at 0 (wrong result)*	#27711	reported

Bug	Report	Status
Join condition statistics narrowed by a sibling condition (debug_verify_stats mismatch)*	#27725	reported
Lambda inside an aggregate inside a lambda: BoundReferenceExpression INTERNAL error*	#27720	reported
SIMILAR TO / regexp filter above a FULL JOIN fails a string_stats assertion*	#27724	reported
IN subquery in JOIN ON plus LEFT JOIN ON subquery: index out of range INTERNAL error*	#27606	duplicate
Outer join max cardinality set to the product: LIMIT 1 returns 2 rows, EXISTS is false*	#27641	duplicate
Correlated IN/NOT IN over mostly-NULL subquery values takes quadratic time*	–	won't report
Query with streaming delim CTEs deadlocks with all threads idle (regression)*	#27737	reported
EXPLAIN (SQL) exports a CSE-hoisted constant as a quantile argument (binder error)*	#27776	reported
DataFusion (closed loop, 20)		
UNION with an EXISTS arm fails: physical input schema differs (stale nullability)	PR #26201	duplicate
GROUP BY c, CAST(c AS BIGINT) fails in eliminate_group_by_constant (duplicate names)	PR #26356	duplicate
ORDER BY ... LIMIT 0 panics in TopK (k > 0) when eliminate_limit does not run	#26431	reported
Correlated EXISTS/IN selecting CAST of its correlated column fails (duplicate names)	#26033	duplicate
ORDER BY col, scalar subquery LIMIT over a RIGHT JOIN fails SanityCheckPlan (regression)	#26572	reported
= ANY (subquery) in a JOIN ON over an aliased table fails: No field named	#26573	reported
Join equivalence drops or keeps side constants wrongly; ORDER BY skipped after RIGHT JOIN	#26036	duplicate
Filter on a constant column is pushed below a global aggregate, returning a wrong row	#26545	duplicate
SanityCheckPlan with sort-merge join when a filter makes a sort or join key constant*	#26624	reported
SanityCheckPlan for two window functions with target_partitions = 1*	#26626	reported
push_down_filter substitutes the wrong expression when col and CAST(col) share a name*	#26619	reported
count(*) OVER () sees only the LIMIT rows under ORDER BY ... LIMIT over another window*	#26617	reported
Panic: two window functions sharing ORDER BY a + 1 get different sort keys (debug)*	#26620	reported
ORDER BY <double> LIMIT n drops -0.0 rows when 0.0 arrives earlier (TopK filter)*	#26618	reported
SanityCheckPlan with sort-merge join when a filter says a + b = a*	#26623	reported
SanityCheckPlan for UNION ALL ... ORDER BY when each arm selects a column twice*	#26625	reported
GROUP BY CAST(a AS VARCHAR), a with a common subexpression fails in optimize_projections*	#26621	reported
x > ALL (subquery) in the SELECT list fails with a schema error unless aliased*	#26622	reported
ORDER BY a * 3 > 1 LIMIT n over a join fails SanityCheckPlan (Null-typed range)*	#26430	duplicate
Sort-merge join claims the right side's ordering when the left key is constant*	#25052	duplicate

Bug	Report	Status
binaryen (compiler trial, 18)		
--translate-to-exnref produces invalid IR for a delegate target ending in a none-typed instr	binaryen#9291	reported
Binary reader: element segment indexes off by one after a declarative segment	–	not yet reported
Text parser leaves declarative element segments out of the elem index space	–	not yet reported
Text parser rejects a table init expression written as plain instructions	–	not yet reported
memory.copy between 32-bit and 64-bit memories aborts the reader	–	not yet reported
Readers reject an if with an unreachable condition targeted by a branch inside it	–	not yet reported
Readers reject an if with block parameters after unreachable	–	not yet reported
Readers abort with length_error on a multivalue if with an unreachable condition	–	not yet reported
Readers hit an assertion in fixExtraOutput for a loop with params	–	not yet reported
Text parser rejects a labeled if whose condition sets a non-nullable local	–	not yet reported
OptimizeInstructions treats two identical suspends as equal (silent miscompile)	–	not yet reported
--outlining treats instructions with different operand types as equal	–	not yet reported
--outlining outlines multi-result sequences without multivalue	–	not yet reported
--outlining aborts on unreachable atomic.rmw, struct.get, call_ref	–	not yet reported
--outlining asserts in StackSignature on repeated code with unreachable operands	–	not yet reported
--translate-to-exnref drops a typed try's type on try~delegate	–	not yet reported
--llvm-nontrapping-fptoint-lowering asserts on an unreachable trunc_sat operand	–	not yet reported
--memory64-lowering makes an unreachable memory.grow/table.grow invalid	–	not yet reported
OCaml (compiler trial, 2)		
Non-exhaustive integer match crashes instead of raising Match_failure	ocaml#15185	reported
amd64 -linscan: Atomic.compare_and_set returns true without storing the value	ocaml#15189	reported
OxCaml (compiler trial, 10)		
-Oclassic crashes on a division by constant zero inside a try	oxcaml#7668	fixed
-Oclassic crashes on a float# field read in an unreachable match arm	oxcaml#7683	reported
Reaper with phantom lets: an unused local in a closure breaks compilation	oxcaml#7684	reported
Binary emitter encodes 32-bit bswap of r8d-r15d without REX.B	oxcaml#7700	reported
-use-ssa: validator rejects x <= 1 against x < 2 for the same switch	oxcaml#7701	reported
-use-ssa: a call before an unused pure op becomes a tail call in one pipeline only	oxcaml#7702	reported
-flambda2-expert-inline-effects-in-cmm: duplicate binding order on a twice-used float	oxcaml#7704	reported
-flambda2-expert-inline-effects-in-cmm: Array.iter2 crash in emit_expr_raise	oxcaml#7705	declined
-no-cfg-eliminate-dead-trap-handlers: regalloc check fails on a nested try	–	won't report
-no-cfg-eliminate-dead-trap-handlers: dead loop around a try kept as a cycle	–	won't report

[*] Found by a later ephyra-agent run (2026-10-21 or 2026-10-22), not in the campaign of Section 4.2 (72 bugs).
[†] Found in review, not by a fuzzing run (6 bugs).